

This is unreleased documentation for Serverpod **Next** version.

For up-to-date documentation, see the [latest version](#) (4.0.0).

3..16.1

WORKING WITH MODELS

 Copy as Markdown

Working with models

A data model is a YAML definition that becomes a typed Dart class on both the server and the client, and, with a [table](#) key, a database table as well. Models are the unit of data your endpoints pass and your database stores. You can also [define models that you only use in Flutter](#).

The recommended file extension is `.spy.yaml` (.spy stands for "Serverpod YAML"), with `.spy` and `.spy.yml` accepted as well. These files can be placed anywhere in your server's `lib` directory, and the extension enables syntax highlighting through the [Serverpod Extension](#) for VS Code. Files with a plain `.yaml` or `.yml` extension are not model files. Serverpod 4 ignores them in `lib/src/models` and `lib/src/protocol` and reports an error asking you to rename them.

The Serverpod CLI reads the model files when generating code and creating migrations. With `serverpod start` running, saving a model file regenerates the code. Outside a session, run `serverpod generate`.

This page covers the model file format: classes, exceptions, enums, default values, and the generated Dart code. The other pages in this group build on it:

- [Field types](#): YAML types mapped to Dart, Postgres, SQLite, and JSON.
- [Inheritance and polymorphism](#): share fields between models with `extends` and `sealed`, and use parent types in endpoints.
- [Custom serialization](#): pass hand-written Dart classes through endpoints and models.
- [Shared packages](#): define models in a package that both the server and the app depend on.

Class

```
class: Company
fields:
  name: String
  foundedDate: DateTime?
```



```
employees: List<Employee>
```

Supported types

The following types can be used as field types. See [Field types](#) for the YAML-to-Dart, Postgres, SQLite, and JSON mapping of each type, including defaults and query operators.

- **Core Dart types:** [bool](#), [int](#), [double](#), [String](#), [Duration](#), [DateTime](#), [ByteData](#), [UuidValue](#), [Uri](#), and [BigInt](#).
- **Vector and geography types:** [Vector](#), [HalfVector](#), [SparseVector](#), [Bit](#), and the [geography types](#).
- **Your own types:** other serializable [classes](#), [exceptions](#), and [enums](#).
- **Collections:** [Lists](#), [Maps](#), and [Sets](#) of the supported types, with the type arguments specified. All supported types can also be used inside [Records](#).
- **dynamic:** holds any serializable value when the type is not known at compile time.

Null safety is supported: append [?](#) to any type to make the field nullable. Once your classes are generated, you can use them as parameters or return types to [endpoint methods](#).

When values are sent between the server and the client, [bool](#), [int](#), [double](#), and [String](#) pass through as native JSON. Other types convert:

Type	Sent as
DateTime	ISO 8601 string, converted to UTC
Duration	Integer, in milliseconds
ByteData	Base64-encoded string
UuidValue	UUID string
Uri	String
BigInt	String

The [Field types](#) catalog lists every mapping, including enums, collections, records, vectors, and geography.

Required fields

Whether a field is required follows from its nullability. Non-nullable fields are always required constructor parameters. Append [?](#) to the type, e.g. [String?](#) or [List<Employee>?](#), to make a field optional. The [required](#) keyword can only be used on nullable fields, and makes the field a required constructor parameter while keeping its type nullable.

```

class: Person
fields:
  name: String           # Required, cannot be null
  nickname: String?, required # Required, but can be set to null
  age: int?              # Optional

```

In the example above, `name` and `nickname` are both required constructor parameters, and only `nickname` accepts null.

TIP

Once your app is in users' hands, changing or removing a model's fields can break older app versions. Adding new fields is safe when they are nullable or have a default value. See [backward compatibility](#).

Limiting visibility of a generated class

By default, generated code for your serializable objects is available both on the server and the client. You may want to have the code on the server side only. E.g., if the serializable object is connected to a database table containing private information.

To make a serializable class generated only on the server side, set the `serverOnly` property to true.

```

class: MyPrivateClass
serverOnly: true
fields:
  hiddenSecretKey: String

```

It is also possible to set a `scope` on a per-field basis. By default all fields are visible to both the server and the client. The available scopes are `all`, `serverOnly`, `none`. A field with a scope other than `all` must be nullable.

INFO

The `none` scope is not typically used in Serverpod apps. It is intended for the Serverpod framework itself.

```

class: SelectivelyHiddenClass
fields:
  hiddenSecretKey: String?, scope=serverOnly
  publicKey: String

```

INFO

Models can be saved to and read from the database. See the [Database](#) section.

Using models on the client only

You can define models that you only use in the Flutter app, for example to hold local UI state or form data. Leave `serverOnly` unset so the class is generated for the client. There is no separate `clientOnly` flag.

Place the `.spy.yaml` file anywhere in the server's `lib` directory. With `serverpod start` running, saving the file regenerates the code; otherwise run `serverpod generate`. Then import the class from the client package.

```
class: CartItem
fields:
  productId: String
  quantity: int
```

```
import 'package:your_client/your_client.dart';

var item = CartItem(productId: 'sku-1', quantity: 2);
```

The class is also generated on the server, which is harmless: without a `table` key no database table is created, and nothing requires you to use the class in an endpoint. Set `immutable: true` if you want value equality for Flutter state management. See [Immutable classes](#).

If you want the YAML and generated classes in a package that both the server and the app depend on, use a [shared package](#).

JSON key aliasing

By default, fields are serialized to JSON using their Dart field name as the key. The `jsonKey` property allows you to specify a different key name for JSON serialization and deserialization, which is useful when integrating with external APIs that use different naming conventions.

```
class: User
fields:
  displayName: String, jsonKey=display_name
  emailAddress: String, jsonKey=email
  createdAt: DateTime, jsonKey=created_at
```

This generates a class where the Dart field names remain camelCase, but the JSON representation uses the specified keys:

```
// Dart field names
var user = User(
```

```

displayName: 'John Doe',
emailAddress: 'john@example.com',
createdAt: DateTime.parse('2024-01-15T10:30:00.000Z'),
);

// Serializes to JSON with custom keys
// {
//   "display_name": "John Doe",
//   "email": "john@example.com",
//   "created_at": "2024-01-15T10:30:00.000Z"
// }

```

This is particularly helpful when:

- Consuming external APIs that use snake_case or other naming conventions
- Working with legacy systems that have specific JSON field requirements
- Working with databases or services that reserve key names, such as mapping `id` to `_id` for MongoDB

INFO

The `jsonKey` property affects JSON serialization and deserialization. It does not affect the database column name. To customize the database column name, use the `column` property instead.

Immutable classes

By default, generated classes in Serverpod are mutable, meaning their fields can be changed after creation. However, you can make a class immutable by setting the `immutable` property to `true`. Immutable classes are especially useful when working with state management solutions or when you need value-based equality.

```

class: ImmutableUser
immutable: true
fields:
  name: String
  email: String

```

When you mark a class as immutable:

1. **All fields become final:** Fields cannot be reassigned after the object is created
2. **Generates `operator ==`:** Provides deep equality comparison between instances
3. **Generates `hashCode`:** Ensures instances with the same values have the same hash code
4. **Compatible with `copyWith`:** You can still create modified copies of immutable objects using the `copyWith` method

Example usage:

```

var user1 = ImmutableUser(name: 'Alice', email: 'alice@example.com');
var user2 = ImmutableUser(name: 'Alice', email: 'alice@example.com');

// Equality comparison works based on values
print(user1 == user2); // true

// Fields are final and cannot be reassigned
// user1.name = 'Bob'; // This would cause a compile error

// Use copyWith to create modified copies
var user3 = user1.copyWith(name: 'Bob');
print(user3.name); // Bob
print(user3.email); // alice@example.com

```

Default values

Fields can be given default values with three keywords that determine where the default applies:

- **default**: sets the default value for both the model (code) and the database (persisted data). It acts as the fallback when the more specific keywords are absent.
- **defaultModel**: sets the default value for the model only, overriding **default** on the code side.
- **defaultPersist**: sets the default value for the database column only, overriding **default** on the persisted side. It is only meaningful on models with a [table](#).

You can use the keywords individually or in combination.

INFO

A database default only applies when an insert omits a value for the column. A field with **default** or **defaultModel** always has a value when the row is written, so its **defaultPersist** never fires. **defaultPersist** only comes into play when the model does not provide a value, for example on a nullable field without a model-side default.

Supported default values

Type	Allowed values	Example
bool	true or false	<code>boolDefault: bool, default=true</code>
int	Any integer value	<code>intDefault: int, default=10</code>

Type	Allowed values	Example
<code>double</code>	Any double value	<code>doubleDefault: double, default=10.5</code>
<code>String</code>	Any string value	<code>stringDefault: String, default='This is a string'</code>
<code>DateTime</code>	<code>now</code> , or a UTC string in the format <code>yyyy-MM-dd'T'HH:mm:ss.SSS'Z'</code>	<code>dateTimeDefault: DateTime, default=2024-05-01T22:00:00.000Z</code>
<code>Duration</code>	A duration in the format <code>Xd Xh Xmin Xs Xms</code>	<code>durationDefault: Duration, default=1d 2h 10min 30s 100ms</code>
<code>UuidValue</code>	<code>random</code> , <code>random_v7</code> , or a UUID string such as <code>'550e8400-e29b-41d4-a716-446655440000'</code>	<code>uuidDefault: UuidValue, default=random</code>
<code>Uri</code>	Any valid URI string	<code>uriDefault: Uri, default='https://serverpod.dev'</code>
<code>BigInt</code>	Any integer value, as a string	<code>bigIntDefault: BigInt, default='1234567890'</code>
Enums	Any of the enum's values	<code>enumDefault: ByNameEnum, default=byName1</code>

For `UuidValue`, `random` generates a UUID v4 (`Uuid().v4obj()` in Dart, `gen_random_uuid()` in the database), and `random_v7` generates a UUID v7 (`Uuid().v7obj()` in Dart, a generated `gen_random_uuid_v7()` function in the database).

For enums, the persisted value follows the enum's serialization mode. A `byName` enum default is stored as the value's name, and a `byIndex` enum default is stored as its index, e.g. `0` for the first value.

On an `int` field, `defaultPersist=serial` hands the value over to a database sequence, the same mechanism behind an `int id` column, though each serial column gets its own sequence. Because only the database can produce it, `serial` is rejected with `default` and `defaultModel`. On SQLite, it is only supported on the `id` column due to dialect limitations.

```
class: Invoice
table: invoice
fields:
  invoiceNumber: int?, defaultPersist=serial
```

INFO

On an `immutable` class, or a class extending one, `default` and `defaultModel` cannot use the non-constant values `now`, `random`, or `random_v7`. On a persisted model, use `defaultPersist` for those instead.

Example

```
class: DefaultValue
table: default_value
fields:
  ### Sets the current date and time as the default value.
  dateTimeDefault: DateTime, default=now

  ### Defaults to false in code and true in the database.
  boolDefault: bool, defaultModel=false, defaultPersist=true

  ### Sets the database-side default value for an integer field.
  intDefault: int?, defaultPersist=20

  ### Defaults to 10.5 in code, with a separate database default.
  doubleDefault: double, default=10.5, defaultPersist=20.5
```

Exception

Model files can define exceptions that can be thrown in endpoints, by using the `exception` keyword. For a more in-depth description of how to work with exceptions, see [Error handling and exceptions](#).

```
exception: MyException
fields:
  message: String
  errorType: MyEnum
```

Enum

The `enum` keyword defines a custom enum with serialization support.

```
enum: Animal
values:
  - dog
  - cat
```

```
- bird
```

By default the enum is serialized by name. You can also opt into index-based serialization by setting `serialized: byIndex`.

```
enum: Animal
serialized: byIndex
values:
  - dog
  - cat
  - bird
```

The `serialized` keyword has two valid values: `byName` (the default) and `byIndex`. When using `byName` the string literal of the enum is used; when using `byIndex` the index value (0, 1, 2, etc.) is used.

WARNING

Using `byIndex` is fragile: adding, removing, or reordering enum values silently changes the serialized form, which can corrupt persisted data. Only use `byIndex` when you need to maintain compatibility with existing data already serialized that way.

Handling unknown enum values

A default value is used when an unknown value is deserialized. This can happen, for example, if a new enum option is added and older clients receive it from the server, or if an enum option is removed but the database still contains the old value.

To configure a default value, use the `default` keyword.

```
enum: Animal
serialized: byName
default: unknown
values:
  - unknown
  - dog
  - cat
  - bird
```

In the example above, if the Enum `Animal` receives an unknown option such as `"fish"` it will be deserialized to `Animal.unknown`. This is useful for maintaining backward compatibility when changing the enum values.

WARNING

If no default value is specified, deserialization of unknown values will throw an exception. Adding a default value prevents these exceptions, but may also hide real issues in your data. Use this feature with caution.

Enhanced enums with properties

Serverpod supports enhanced enums that can have custom properties attached to each value. This is useful when you need to associate additional data with enum values, such as display names, codes, or configuration values.

To define an enhanced enum, add a **properties** section that declares the property names and types, then specify property values for each enum value:

```
enum: HttpStatus
serialized: byName
properties:
  code: int
  message: String
values:
  - ok:
    code: 200
    message: 'OK'
  - notFound:
    code: 404
    message: 'Not Found'
  - internalError:
    code: 500
    message: 'Internal Server Error'
```

This generates an enhanced Dart enum with the specified properties:

```
enum HttpStatus implements SerializableModel {
  ok(200, 'OK'),
  notFound(404, 'Not Found'),
  internalError(500, 'Internal Server Error');

  const HttpStatus(this.code, this.message);

  final int code;
  final String message;

  // Serialization methods...
}
```

You can then access the properties on any enum value:

```
var status = HttpStatus.ok;
print(status.code);    // 200
```

```
print(status.message); // OK
```

Supported property types

Enhanced enum properties support the following types:

- `int` / `int?`
- `double` / `double?`
- `bool` / `bool?`
- `String` / `String?`

Default property values

Properties can have default values, making them optional when defining enum values. The syntax follows the same pattern as [class field defaults](#):

```
enum: Priority
properties:
  level: int
  description: String, default='No description'
values:
  - low:
    level: 1
  - medium:
    level: 2
    description: 'Medium priority'
  - high:
    level: 3
    description: 'High priority - handle first'
```

In this example, `low` will use the default description `'No description'`, while `medium` and `high` have explicit descriptions.

Adding documentation

Serverpod allows you to add documentation to your serializable objects in a similar way that you would add documentation to your Dart code. Use three hashes (###) to indicate that a comment should be considered documentation.

```
### Information about a company.
class: Company
fields:
  ### The name of the company.
  name: String

  ### The date the company was founded, if known.
```

```
foundedDate: DateTime?
```

```
### A list of people currently employed at the company.
```

```
employees: List<Employee>
```

Generated code

Serverpod generates some convenience methods on the Dart classes.

copyWith

The `copyWith` method creates a copy of an object with selective field updates and is available on all generated classes:

```
// User has a nullable age field: age: int?  
var john = User(name: 'John Doe', age: 25);  
var jane = john.copyWith(name: 'Jane Doe'); // age stays 25  
var ageless = john.copyWith(age: null);      // age is explicitly set to null
```

The `copyWith` method generates a deep copy of an object, preserving all original fields unless explicitly modified. As the example shows, it distinguishes between a nullable field set to `null` and a field left unspecified.

toJson / fromJson

The `toJson` and `fromJson` methods are generated on all models to help with serialization. Serverpod manages all serialization for you out of the box and you will rarely have to use these methods yourself. See [Custom serialization](#) for more info. On the server, models also get a `toJsonForProtocol` method that produces the JSON sent to clients. It leaves out fields whose `scope` hides them from the client.

Custom methods

Sometimes you will want to add custom methods to the generated classes. The easiest way to do this is with Dart's extension feature.

```
extension MyExtension on MyClass {  
  bool isCustomMethod() {  
    return true;  
  }  
}
```

Related

- [Field types](#): YAML types mapped to Dart, Postgres, SQLite, and JSON.

- **Model reference:** every keyword available in a model file, and whether it applies to a **class**, **exception**, or **enum**.
- **Tables:** store a model in the database.
- **Backward compatibility:** evolve models without breaking older app versions.